Fundamentals of Machine Learning
Understanding Mock: Solutions

For every answer: the verdict, the mechanism, the evidence that would confirm it, the fix, and the tempting wrong answer

Fundamentals of Machine Learning

2026-10-31

1. The phishing detector that was too good

(a) Anything that differs between *the 2015 corpus* and *2026 company mail*, whatever the email's intent: (1) the `Date` header (year 2015–2017 versus 2026), a perfect separator on its own; (2) the `X-Mailer` and the HTML templates (old mail clients and webmail kits versus the company's 2026 Outlook template and signature); (3) vocabulary of the period and of the company (product names, colleagues' names, internal jargon only in legitimate mail; 2015 brands and topics only in phishing). The model only needs one of these to reach 99%. It never has to learn what makes a message deceptive.

(b) No: it is a **shortcut**, not leakage. Leakage means a feature that is *not available at decision time*, or that is computed from the label or from the future (for example a "reported by user" flag). The `Date` header *is* available when the email arrives. The problem is that, *because of how the data were collected*, it is correlated with the label in training, and that correlation does not exist in deployment, where all mail is from 2026. Leakage is fixed by removing information the deployed system cannot have. A shortcut is fixed by changing the collection so the spurious cue stops predicting the label.

(c) Test on data where the source and the label are **no longer confounded**. Hand-label a few hundred 2026 phishing emails that reached the company. Evaluate on them (all 2026, all from the same servers) together with held-out 2026 legitimate mail. If the model detects phishing, recall stays high. If it detects the source, it calls the 2026 phishing "legitimate" and recall collapses. Two complementary checks: retrain without the header features and watch the validation accuracy fall; or train a classifier to predict the *source* from the features. If it is near-perfect, the source is learnable and is a candidate shortcut.

(d) High accuracy shows that the features *predict the label in this dataset*. It shows neither causation nor that the relation holds elsewhere. Here the date predicts the label perfectly and obviously does not make an email phishing. The deployment failure is the proof. A model learns *any* cue correlated with the label in the training distribution; whether the cue is stable is a property of the data collection, not of the accuracy.

(e) Collect both classes **from the same source and period**: phishing that reached *this* company in 2026 (user reports, quarantine), plus legitimate mail from the same weeks. Also drop or normalise the headers that encode the collection process. Ideally make the training distribution match the deployment distribution, then evaluate on a later period.

Shape: S1 audit, S3 shortcut versus leakage, S5 experiment. **Course:** Essential ch.1 "Generalization and shortcuts"; T1 1e.

Tempting wrong answer. "The model overfits, add regularisation." Regularisation does not help: the shortcut generalises *perfectly* to the validation set, because the validation set has the same confound.

2. A tree that cannot reach zero training error

(a) A fully grown tree can put every *distinct* feature vector in its own leaf. The only error it cannot remove is between **rows with identical feature vectors x and different labels**: such rows always fall in the same leaf, and the leaf predicts their majority. So 15% of the training rows are the minority label inside groups of identical x. (With continuous features this means many flows are exact duplicates in the representation, which is plausible: short flows with the same port, protocol and packet counts.)
(b) Not necessarily. *It might*: annotation errors produce conflicting labels on identical x. *It might not*: the labels can be correct and the **representation too poor**. A flow labelled "attack" because it belongs to an attack episode can look exactly like a benign flow (T1 1b). The label depends on information that is not in x. Check: look *where* the conflicting rows are. If they are concentrated in the benign-looking flows of known attack episodes (the same SYN-ACK handshake, the same small HTTP request), the labels are right and the representation is missing context. If they are spread at random, with no relation to episodes, re-examine a sample by hand: that is label noise.
(c) Training error stays ≥ 0.15: no function of x separates identical x, so even 5,000 deep rounds cannot. The booster spends its capacity fitting noise in the other 85%, so validation error stays near 0.17 or gets *worse*. More capacity attacks estimation error, and here the dominant error is the Bayes error of this representation.
(d) Only a **change of representation**: give x the information that distinguishes the cases. Adding features can never increase the Bayes error (a finer x can always ignore the new part), and it can lower it. Choosing a model works *within* a fixed x, so it cannot go below the Bayes error of that x. Example: window features of the source host (T1 P8), such as the number of distinct destination ports in the last 60 s, the failure fraction, or the episode context. Two identical flows then differ by what surrounds them.
(e) `flow_id` makes every row distinct, so the tree can memorise each label: training error 0. This is capacity without generalisation. *Random split:* validation ids lie between training ids, and neighbouring ids are often the same episode, so the tree learns "ids in this interval are attacks" and validation error may even *improve*, falsely: an ordering leak. *Temporal split:* validation ids are larger than every training id, all fall in the last leaf interval, and the feature is useless or harmful.

Shape: S2 does it necessarily mean, S4 diagnose. **Course:** Essential ch.2 "Representation and unavoidable ambiguity", ch.3 capacity.

Tempting wrong answer. "The tree is not deep enough." It is fully grown; depth is not the bottleneck.

3. Four learning curves, four stories

(a) Team A: underfitting (high bias), not data-limited. Train ≈ validation means low variance. Both being high and flat in n means more data will not help: the model family (or the representation) cannot express the relation. *Do:* increase capacity (deeper trees, interactions, less regularisation) or add better features. If even a very flexible model stays near 0.30 on *training* data, suspect Bayes error from the representation (Problem 2).
(b) Team B: high variance, data-limited regime. The large train/validation gap means the fit depends heavily on the sample. A validation curve still falling steeply means more data will keep helping. *Do:* collect more data if possible. Otherwise reduce variance with regularisation, a smaller capacity, bagging or a random forest, or early stopping. Tune by validation, not by training error.
(c) Team C: distribution shift, not a learning failure. With a fixed model, more data from *the same distribution* should not make validation error grow. So February and March differ from January: new services, a changed attack mix, or a change in the labelling process. The model now fits a mixture and moves away from January. *The real question:* which distribution

matters? Deployment is in April. If the model is getting closer to April traffic, the *validation set* is the problem. Use a temporal split (validate on the most recent month), compare error month by month, and check whether the label definition or tool changed.

(d) Team D: validation below training is a warning sign, not a success. Possible mechanisms: *sampling noise*: with 400 flows the standard error of an error near 0.1 is about $\sqrt{0.1 \times 0.9/400} \approx 0.015$, so a gap of 0.03 is about two standard errors, unusual but possible. An *easier validation set*: drawn from a quieter period, or after hard cases were removed. *Duplicates* of easy training flows in validation. Strong regularisation makes the model *biased*, but a biased model is still evaluated on its own training data. Regularisation alone does not make training error larger than validation error *on average*. *Do:* use a larger or cross-validated estimate, check the composition of the validation set (class balance, period, duplicates).

Shape: S4 diagnose from results. **Course:** Essential ch.3 "Honest evaluation: learning curves"; review Q20.

Tempting wrong answer. For (c): "more data causes overfitting". More data never increases variance. Something about the data changed.

4. Same model, new company, useless alerts

(a) Prior shift. Recall (TPR) and FPR are computed *within* a class: $\mathbb{P}(\text{alert} \mid Y = 1)$ and $\mathbb{P}(\text{alert} \mid Y = 0)$. They depend only on $p(x \mid y)$, which is unchanged. Precision mixes the classes:

$$\text{precision} = \frac{\pi\,\text{TPR}}{\pi\,\text{TPR} + (1-\pi)\,\text{FPR}}.$$

With TPR $= 0.8$ and FPR $= 0.01$: at $\pi = 0.05$, precision is $0.04/0.0495 = 0.81$; at $\pi = 0.005$, precision is $0.004/0.01395 = 0.29$. Same detector, ten times fewer attacks, so roughly three false alerts for every true one.

(b) Not necessarily. If only the prior changed ($p(x \mid y)$ the same), the model's class-conditional knowledge is still valid. Correct the posterior with the prior ratio (the T1 6e formula), or equivalently raise the threshold, so the alert rule matches Q's prior and costs. No retraining is needed under the **prior-shift assumption**. If attacks or benign traffic look different at Q, it is not enough.

(c) Under pure prior shift, the distribution of scores at Q must be a **mixture of the same two class-conditional score distributions** seen at P, only with a different weight π_Q. You can fit that weight without labels (for example with EM on the scores). Evidence against prior shift: score mass in ranges where neither class had any at P; input values never seen at P (new ports, new services, new byte ranges), which a generative model flags as low $p(x)$; or a score histogram no choice of mixture weight can reproduce. Then the data themselves changed (covariate or concept shift), and you need labels from Q.

(d) Training on 50/50 data makes the model estimate the posterior *under a prior of 0.5*. Its outputs are then systematically too high for Q, where π is about 0.005. Every score must be corrected back with the prior-ratio formula, or the precision problem gets *worse*. Resampling changes the *prior* the probabilities refer to, which is a statement about the data. The cost threshold $C_{FP}/(C_{FP} + C_{FN})$ changes the *decision* taken from correct probabilities, which is a statement about consequences. The two do different jobs, and confusing them double-counts or cancels effects (T1 2d, 6f). Resampling can still help *estimation* when attacks are very rare (T1 6g), but it must be undone afterwards.

(e) A fixed budget turns the rule into **rank and take the top 50 per day**. The effective threshold is whatever score the 50th flow has. Only the *ordering* of the scores matters: any increasing transformation of the scores picks the same 50 flows, so calibration does not affect *which* flows are investigated. It still matters for reporting the expected number of missed attacks, or for deciding whether the budget is enough. The metric to optimise becomes precision (or recall) at the top k, not accuracy or log loss.

Shape: S3 distinguish (prior shift / costs / resampling), S6 deployment. **Course:** Essential ch.4 "Class imbalance, prior shift"; ch.2 Bayes rule; T1 P6.

Tempting wrong answer. "The model got worse at Q." It did not: TPR and FPR are the same. Precision depends on the prior.

5. When probabilities lie

(a) The logistic score $s(x) = w^\top x + b$ is linear and *unbounded*. Far along a direction with $w_j > 0$, the score keeps growing and $\sigma(s) \to 1$. The model extrapolates its training boundary to a region it never saw, and the posterior there is pure extrapolation. A discriminative model models $p(y \mid x)$ only; nothing in it represents "how typical is x". High confidence measures *distance from the decision boundary*, not familiarity.
(b) A generative model also gives $p(x) = \sum_c \pi_c\, p(x \mid c)$. A 40 GB flow has a tiny density under both class models, so you can flag inputs with $p(x)$ below a level seen on the training data as *out of distribution*. (This is only as good as the density model, but it is a signal the discriminative route does not have.) Operationally, use the reject option: send the flow to human review (T1 2c) instead of trusting the 0.995, and log it as a candidate for new training data.
(c) Accuracy is measured at threshold 0.5, but the rule uses $t = 2/(2+8) = 0.2$. So the 0.95 versus 0.93 says little about the decisions you will actually make. Log loss rates the probabilities *at all levels*, which matters when you threshold at 0.2: B's better log loss suggests its probabilities are more reliable. The decisive comparison: apply each model's rule at $t = 0.2$ on the validation set and compare the **average cost**, $2 \cdot \mathrm{FP} + 8 \cdot \mathrm{FN}$ per flow. Choose by the loss you will be judged on.
(d) Naive Bayes multiplies the likelihood ratios of the features as if they were independent given the class. Ten copies of the same byte evidence get **counted ten times**: the log-odds are roughly ten times too large, which pushes posteriors to 0 or 1. *Accuracy* may hardly suffer, because inflating all log-odds by the same factor keeps the sign, so decisions at 0.5 are similar. *Cost- sensitive decisions* suffer: with every posterior at 0.0001 or 0.9999, the threshold 0.2 is no longer meaningful. Fix by removing redundant features, or by calibrating on validation data.
(e) Only if the model is **calibrated**: among all flows that receive $\hat{p} \approx 0.7$, about 70% really are attacks. Cross-entropy training aims at this, but capacity, imbalance handling, shift and Naive Bayes double counting all break it. Check on held-out data: group flows by predicted probability and compare the mean prediction with the observed attack fraction in each group (a reliability diagram).

Shape: S3 distinguish, S6 trust. **Course:** Essential ch.4 "Comparing the two routes", "Unfamiliar inputs"; ch.2 evaluation; T1 7(II)c.

Tempting wrong answer. For (c): "A, because accuracy is higher." The decision does not use the threshold accuracy was measured at.

6. Ensembles that do not behave as advertised

(a) Bagging reduces **variance** only: averaging M models with variance σ^2 and correlation ρ gives variance $\rho\sigma^2 + (1-\rho)\sigma^2/M$. A logistic regression is a *stable* learner: refitting on bootstrap samples gives almost the same model, so there is very little variance to remove ($\rho \approx 1$). A deep tree is *unstable*: each bootstrap sample gives a very different tree (low bias, high variance), so averaging removes a lot.
(b) *Reason 1, near-duplicates:* a flow that is out of bag for a tree usually has near-identical twins from the same episode *in* that tree's bootstrap sample. The OOB estimate then behaves like a random row split and is optimistic. *Reason 2, time shift:* the test month has different traffic (new services, new attacks), which no estimate from the training months can see. A validation set **grouped by episode** removes reason 1 (the twins land on the same side). Only a **temporal** split estimates reason 2.

(c) In a forest the trees are built *independently* and averaged. More trees only make the average converge to its expectation, so the variance term decreases with M and nothing new is fitted; M is not a capacity knob. In boosting, each tree fits the *residuals of the training set*, so every round adds fit. After round 200 it is fitting noise in the training sample, and capacity grows with every round. *Do:* early stopping on validation loss (keep the round of the minimum), a smaller learning rate with more rounds, shallower trees, subsampling, or larger λ and γ.
(d) The ephemeral source port is random: it should carry no information about attacks. **MDI is computed on the training data and favours features with many distinct values**: a feature with thousands of possible cut points can always find one that reduces training impurity a little, even by chance, and deep trees add those small reductions up. *Check:* permutation importance on *validation* data, where it should be close to 0. A further test: add a pure random column and see whether MDI also ranks it highly. If `src_port` survives on validation, look for a real artefact, such as one attack tool that uses a fixed source port.
(e) The meta-model was probably trained on predictions the base models made *on their own training observations*. A deep tree interpolates its training data, so its in-sample predictions look perfect, and the meta-model trusts it. *Fix:* train the meta-model on **out-of-fold** predictions. Split the data into K folds, fit every base model on $K-1$ folds and predict the held-out fold, so each observation gets a prediction from models that never saw it (T2 8b).

Shape: S4 diagnose, S3 distinguish (bagging/boosting, MDI/permutation, OOB/out-of-fold). **Course:** Essential ch.5, all sections; T2 P2, P3, P8.

Tempting wrong answer. For (c): "random forests cannot overfit". Each tree can overfit; it is *adding trees* that cannot add overfitting.

7. Geometry problems

(a) kNN distance adds up *all* squared coordinate differences equally. With 50 noise columns against 8 real ones, the noise dominates the distance, so the "nearest" neighbours are nearly random with respect to the real features. The representation's geometry is ruined (the curse of irrelevant dimensions). A tree chooses each split by impurity reduction, so it mostly picks the real features and ignores the noise. It still loses a little: with $m_{\text{try}} \approx \sqrt{58} \approx 8$ features offered per split, often none of the 8 good ones is offered, so some splits are made on noise; noise splits also appear by chance deep in the trees.
(b) Numeric order gives distances that have nothing to do with meaning. Ports 22 (SSH) and 23 (Telnet) are distance 1 apart, as are 443 (HTTPS) and 445 (SMB), which are unrelated services with different attack profiles. Meanwhile 80 and 8080 (both HTTP) are 8,000 apart. Better: treat the port as **categorical**. One-hot encode the frequent ports, put an "other" category for the rest, or group by service (web, remote access, file sharing) or by range (well-known, registered, ephemeral).
(c) No. 1-NN training error is 0 *by construction*: each training point is its own nearest neighbour (unless identical x carry different labels). It says nothing about generalisation. Over- or underfitting is judged on validation data. Low k does mean high capacity and high variance, and that can be seen from the train/validation gap and from how the error changes as k grows.
(d) *Random split:* each validation flow has a near-twin from the same episode in the training set. The single nearest neighbour is that twin with the same label, so $k = 1$ looks excellent: it is an evaluation leak through duplicates. *Grouped split:* the twins are gone, the nearest flows come from other episodes, single neighbours are noisy, and averaging over 25 helps. The grouped number is the honest one for new episodes, so it is higher.
(e) *kNN:* scaling defines the geometry, so statistics computed with the test set slightly change the distances used, which is a small but real leak. On principle, fit the scaler on training data only, and inside each fold for cross-validation. *Tree:* a split "$x_j < c$" is invariant to any increasing transformation of x_j, so standardisation does not change the tree's predictions at all. The leak

has no effect there. (Missing-value imputation or target-dependent encodings fitted on all data would still leak.)

Shape: S4 diagnose, S1 audit. **Course:** Essential ch.1 representations, ch.3 "Honest evaluation"; T1 P3, 4e.

Tempting wrong answer. For (e): "scaling leakage matters equally for every model". It depends on whether the model uses the geometry.

8. Reading coefficients and selections

(a) A zero L1 coefficient means the feature is **redundant given the others**, not irrelevant. With correlation 0.98, `bytes_received` already carries almost the same information. L1 keeps one feature of a correlated group, and *which one* is nearly arbitrary. On another training sample the choice can flip (selection instability). L2 would share the weight between the two. To ask whether bytes sent matter, fit with `bytes_sent` alone, or look at the group.
(b) No. A coefficient's size depends on the *units*: one millisecond is a tiny change, while `is_port_22` goes from 0 to 1. Duration varies over thousands of ms, so $0.0004 \times 5{,}000 = 2$ log-odds across its range, comparable to the port's 1.9. Compare coefficients only after standardising, and even then correlated features share or steal credit. Importance is better measured by permutation on validation data.
(c) Do not believe it. The 20 features were chosen *using all 60 labels*, including the labels of the hosts that later serve as validation folds. With 8,000 features and 60 hosts, many pure-noise features correlate strongly with the label by chance. Selection picks exactly those, and the CV folds have already been "seen". With coin-flip labels the procedure would still report well above 50% (often 70–90%), while the true accuracy is 50%. *Fix:* put the selection **inside** each fold (select on the $K-1$ training folds only, then evaluate on the held-out fold), or use a separate test set never touched by selection.
(d) Patience is how many rounds without improvement you tolerate before stopping. It separates a real minimum from **noise in the validation curve**. Patience 1: you stop at the first random bump, too early, and underfit. Very large patience: you train long past the minimum, wasting compute (keep the best round, not the last). Also, picking the best round *on* validation makes that validation loss slightly optimistic, so report the test set.
(e) *Right:* beyond the interpolation threshold ($p \approx n$), larger models can generalise better again (double descent), so "bigger is always worse" is false. *Wrong:* the U-curve is still the right description below the threshold and in many regularised settings, where the peak is smoothed out. Near the threshold the error really is high. "Always use the largest" ignores compute and data size, and the location of the threshold moves with n. Model size is still chosen on validation.

Shape: S2 does it necessarily mean, S1 audit. **Course:** Essential ch.3 "Controlling effective capacity", "Selecting features without fooling yourself"; review Q10–14, Q16.

Tempting wrong answer. For (c): "cross-validation was used, so the estimate is fair". CV is fair only if *every* data-dependent step is inside the folds.

9. Compromised accounts and labels that arrive late

(a) (1) **Future information**: "logins in the month of the session" counts logins that happen *after* the session, which is not available at decision time (and a compromise often causes more logins later). (2) **Random split by session**: sessions of the same user and of the same incident end up on both sides, so the model is tested on users and incidents it has already seen. (3) **User-ID embedding**: the model can memorise *which users* were compromised, an identity shortcut. For new April employees, the embedding is untrained and the feature is useless. Each of these three raises the test score without helping on the stated goal.
(b) Investigations take about ten days, so incidents from late March were *not yet labelled* when

the data were extracted. Their sessions carry the default label "benign". The model is taught that recent attack patterns are benign (label delay, i.e. label noise concentrated on the newest, most relevant data). *Handle:* use only sessions whose labels are mature (older than about 10 days, or older than most investigation times), or keep them out of training and re-extract later.
(c) Keep the time order and group by user. Train on January–February using users of group A. Validate (to choose models and thresholds) on mature-labelled March sessions of *other* users, group B. Test once on April sessions of new employees, or simulate them with users held out from all earlier steps. Compute every feature from data available *before* the session (rolling counts over the past days), drop the user-ID embedding, and fit all preprocessing on the training part only.
(d) It measures each session as an isolated decision and ignores **time to detection** relative to the incident. An alert on the last session of a finished attack counts as a true positive but has no value. Measure per *incident*: the fraction of incidents detected, and the delay from the first malicious session to the first alert (as in T1 8b), possibly the number of sessions the attacker had before the alert.
(e) A **feedback loop**, i.e. selection bias in the labels. Attacks the detector misses are never investigated, so they stay labelled benign. Retraining on these labels teaches the model that its blind spots are benign, and the blind spots grow. The measured precision looks good while true recall quietly falls. *Minimal fix:* investigate a small **random sample of non-flagged sessions** every month. This gives unbiased labels for estimating recall, and they can be used, with appropriate weighting, in training.

Shape: S7 decision time, S1 audit, S6 deployment. **Course:** Essential ch.3 "Honest evaluation" (temporal and grouped splits, leakage); ch.1 shortcuts; T1 P8.

Tempting wrong answer. For (b): "late March has fewer attacks". Nothing says it has; the labels have not arrived.